

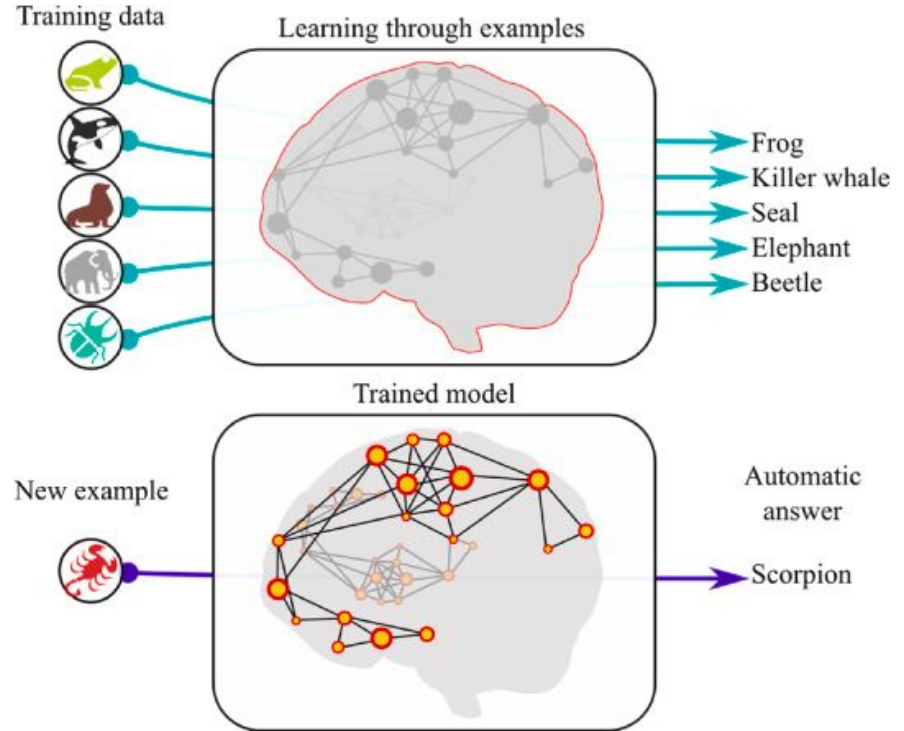
Kaggle #13



Basics and Intro

What is deep learning?

An approach to ML using multi layered artificial neural networks inspired by the human brain to learn complex patterns.



What can Deep Learning Do?

- Image recognition (faces, medical scans, self-driving cars)
- Language translation (ChatGPT, translation, autocomplete)
- Fraud detection
- Game playing
- Protein folding (AlphaFold)

- One thing in common: nobody hand-coded the rules

Deep Learning vs Classical ML?

- Classical ML: you engineer features, model learns a mapping
- Deep Learning: you feed raw data, model learns the features too
- More data + more compute = better results (usually)

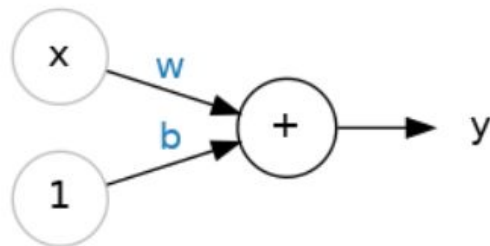
A Neuron (Linear Unit)

Fundamental component of a neural network : a neuron

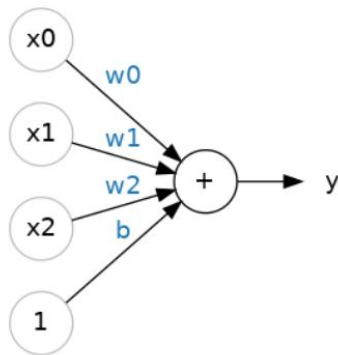
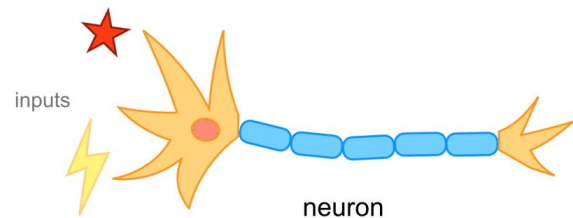
Input: x

Weight : w

B: bias



The Linear Unit: $y = wx + b$



A linear unit with three inputs.

Multiple inputs

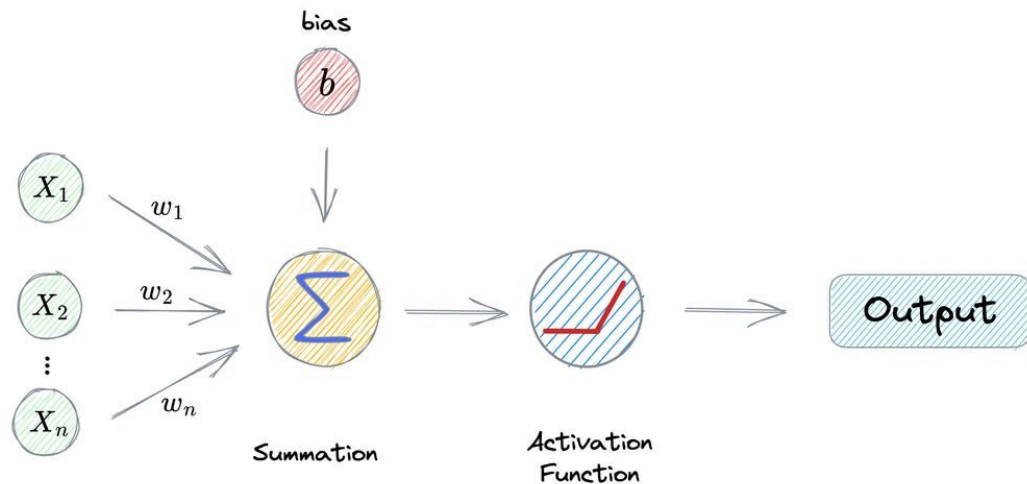
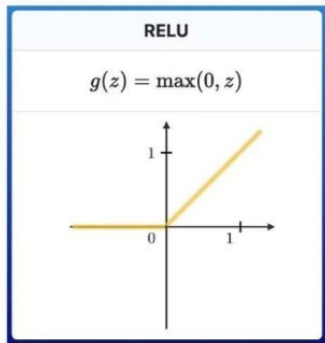
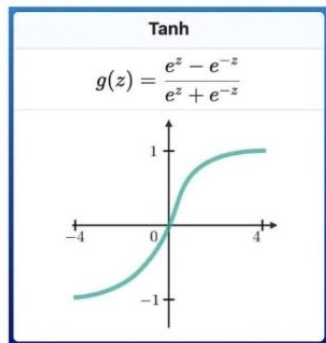


A Neuron (A Description)

- Takes several inputs
- Multiplies each by a weight (importance)
- Adds a bias
- Passes through an activation function
- Outputs a single number
- Analogy: a neuron fires if enough evidence

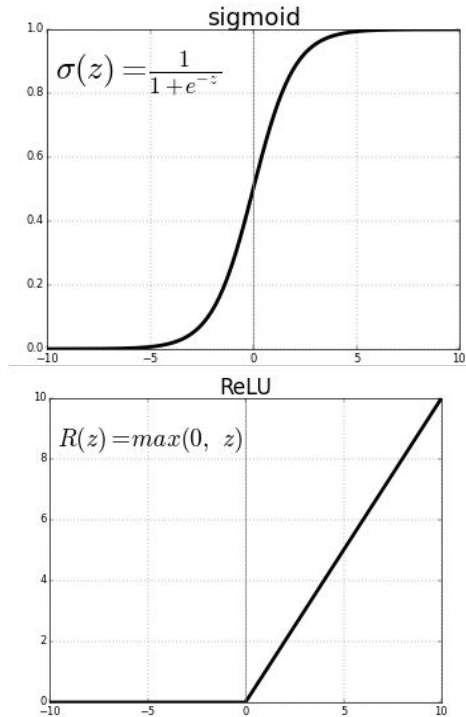
Activation Functions

Mathematical functions applied to a neuron's **output** to introduce non-linearity -> allowing the network to learn complex patterns



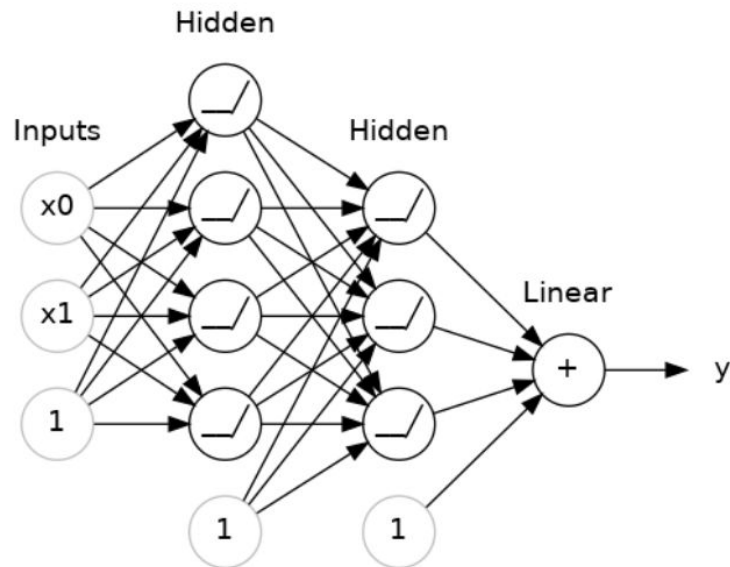
The need for an Activation Function

- Without activation: just a linear function, no matter how many layers
- ReLU: if input < 0 , output 0. Otherwise pass it through. Simple, works well.
- Sigmoid: squashes output between 0 and 1. Used for probabilities.
- We use ReLU in hidden layers
- Sigmoid at the output for binary classification



A Network of Neurons

- Stack neurons into layers: input layer, hidden layers, output layer
- Each layer transforms the data into a new representation
- Early layers: simple patterns. Deeper layers: complex combinations.
- Our network today: 11 inputs $\rightarrow$ 128 $\rightarrow$ 64 $\rightarrow$ 32 $\rightarrow$ 1 output



A stack of dense layers makes a "fully-connected" network.

A Forward Pass

- Data enters the input layer
- Flows through each layer left to right
- Each neuron computes its weighted sum + activation
- Final output: a predicted probability
- No learning yet — this is just a prediction

Loss Function

- We need to measure how wrong the prediction is
- Loss = difference between predicted and actual
- Binary Cross-Entropy: standard loss for classification (you may have seen this before last sem)
- Lower loss = better predictions
- Goal of training: minimise the loss

Stochastic Gradient Descent

- Imagine the loss as a hilly landscape
- We want to find the lowest point (minimum loss)
- Gradient = slope at current position
- Take a small step downhill, repeat
- Learning rate = size of each step.
 - Too big: overshoot
 - Too small: very slow

Your Task now

- We are going to apply this to the March Playground competition.
- (This won't give you a good submission, it's just for understanding)
- You have three tasks to complete

Task 1 -- Architecture

- `nn.Linear(in, out)` -- a fully connected layer
- `nn.ReLU()`, `nn.Tanh()`, `nn.Sigmoid()` -- activation functions
- The `in` of each layer must match the `out` of the previous one

Task 2 - Training loop

```
preds = model(X_batch)      # forward pass
loss = loss_fn(preds, y_batch) # compute
loss                          # loss
optimizer.zero_grad()       # clear old
gradients                    # gradients
loss.backward()              # backprop
optimizer.step()             # update weights
```

Task 3 -- Tuning

- **LEARNING_RATE** -- try **0.01** or **0.0001**
- **EPOCHS** -- more = more training, watch for overfitting
- **Layer sizes, activations, add `nn.Dropout(p=0.3)`**
- **Target: beat 0.85 AUC**

Feedback Form

